
EEEM071 Advanced Topics in Computer Vision and Deep Learning

Lab Week 10

Pytorch Evaluation Metrics

Dr. Xiatian Zhu
xiatian.zhu@surrey.ac.uk

1 Introduction

An evaluation metric is a quantitative measure that is used to assess the performance of a machine learning model. It is used to determine how well a model is performing in terms of accuracy, precision, recall, F1-score, or other relevant metrics.

The choice of evaluation metric depends on the problem domain and the specific goals of the model. For example, if the problem is binary classification, the evaluation metrics commonly used are accuracy, precision, recall, F1-score, and area under the ROC curve (AUC-ROC). On the other hand, for regression problems, evaluation metrics such as mean squared error (MSE), root mean squared error (RMSE), mean absolute error (MAE), and R-squared (R^2) are used.

It is important to select the appropriate evaluation metric that aligns with the business objectives and problem requirements. A good evaluation metric should be interpretable, intuitive, and sensitive to the performance of the model in real-world scenarios.

2 Evaluation for Vehicle Re-identification

In information retrieval (such as documents, images, and music), mean average precision (mAP) is used to evaluate the performance of a search algorithm. It is calculated by measuring the precision and recall of the algorithm for each query and then averaging these values across all the queries.

Cumulative Matching Characteristics (CMC) curves are commonly used to evaluate the performance of vehicle re-identification methods. In this setting, a vehicle's identity is verified by matching them to a single image in a gallery.

To evaluate the performance of a vehicle re-identification algorithm, it ranks all the images in the gallery based on their similarity to the query image. The algorithm then determines the accuracy of the match by

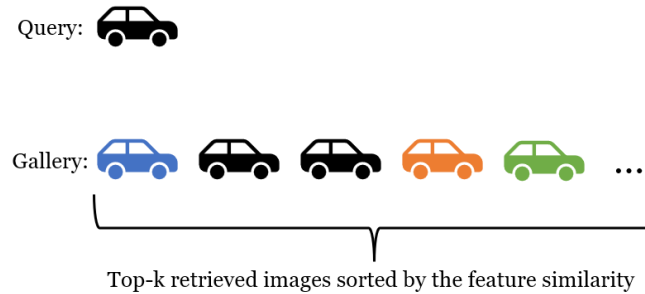


Figure 1: A retrieval result, given a query (black car), we want to retrieve images that have the same identity (other black cars). The retrieved images are sorted based on the feature similarity between the black car and other images.

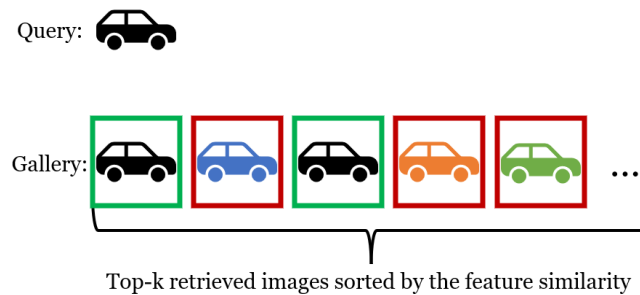


Figure 2: To evaluate whether our algorithm is good or not, we need to see whether the query image matches the retrieved images or not. The green color indicates the correct match and the red color indicates the wrong match. Sometimes correct match also means a true positive and a wrong match means a false positive.

calculating how many of the top-ranked images contain the correct identity:

$$\text{Rank-}k = \begin{cases} 1, & \text{if top-}k \text{ ranked gallery samples contain the query identity} \\ 0, & \text{otherwise} \end{cases}$$

2.1 Implementation

To implement these evaluation metrics, you will need to have the query label and gallery labels. Assuming the two situations above (in Fig. 3 and 4, where there are two queries to search the black car, you have obtained the results below:

```
query_labels = [0, 0]
gallery_labels = [[0, 1, 0, 2, 3], [1, 2, 0, 3, 0]]
```

where `query_labels` is the label of two queries and `gallery_labels` is the label of two sets of retrieved items corresponding to the two queries.

The correct answer is shown in Fig. 3 and 4. If your implementation can get the **same** result as above, then you are managed to evaluate your algorithm for vehicle identification.

You can refer to this [function](#) of the coursework to see how the implementation is done in actual practice.

There is a [library](#) by huggingface that has a lot of implemented evaluation metrics. If you think your implementation of the evaluation metrics is incorrect, it is good to refer to libraries like this.

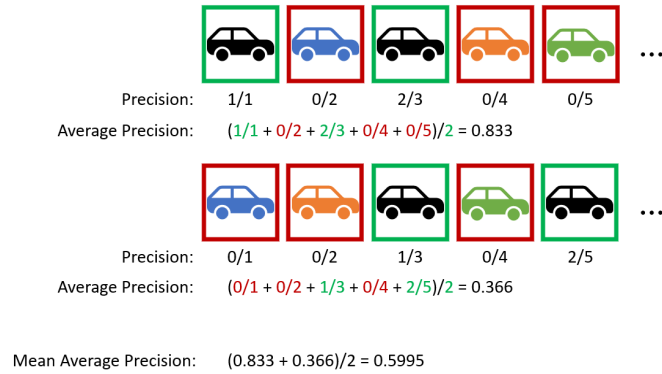


Figure 3: Once we have known the matches, we then can start calculating mean average precision for each query as shown above.

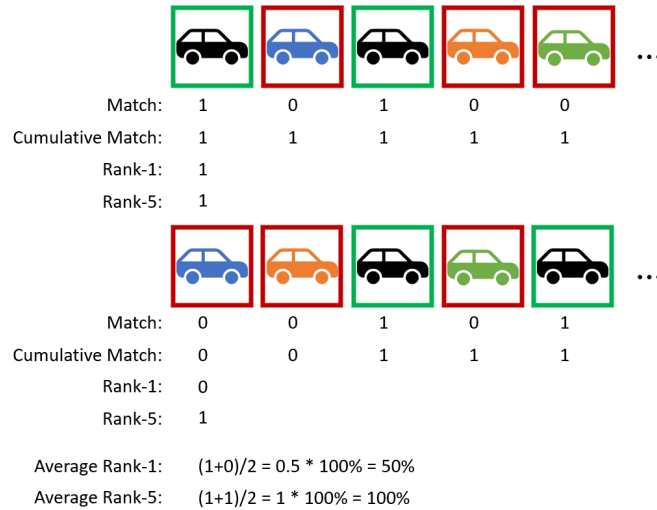


Figure 4: Similarly, the cumulative match characteristic (CMC) is computed as shown above.